CAPSTONE PROJECT PROPOSAL · SECTION B1

CARE

Conflict-free AI Routine Engine

An AI-powered system that generates conflict-free, rule-aware department class routines, balances what teachers and students need, and makes the most of every classroom.

Team CARE

MEMBER	STUDENT ID	SECTION
Sameul Hasan	232-35-334	B1
Jannatul Ferdous Achol	232-35-587	B1

Department of Software Engineering · Daffodil International University · September 2026

1 Project Title

Project title	**CARE: Conflict-free AI Routine Engine**
Course	Capstone Project, Section B1
Team	Team CARE: Sameul Hasan (232-35-334); Jannatul Ferdous Achol (232-35-587)
Department	Software Engineering, Daffodil International University

2 Real-life Problem

Most departments still prepare class routines by hand, which takes days and often produces errors. Common problems include:

- A teacher assigned to two classes at the same time
- The same room booked for two classes, while other rooms sit empty and large rooms hold small batches
- Overlapping classes for one batch, and labs placed in regular rooms
- Department rules (such as maximum classes per teacher per day) missed because they are scattered across documents
- Routines that ignore what teachers and students need: teachers get slots outside their availability, while students face long gaps, overloaded days and classes that don't match university transport times

Any later change, such as a teacher's absence or a makeup class, means redoing much of the work manually.

3 Proposed Solution

We will build a web-based system that automatically generates a conflict-free, rule-aware and fair class routine for the whole department. It has five main parts.

1. AI-based scheduler

A genetic algorithm searches thousands of possible routines. It always satisfies hard constraints (no teacher, room or batch clashes; labs in lab rooms; room capacity) and optimizes soft constraints (fewer gaps, balanced daily load, preferred slots).

2. RAG-based rule understanding

The admin uploads department policy documents. The system retrieves the relevant rules and uses an AI model to convert them into structured scheduling constraints, which take effect only after admin approval.

3. Fair balancing of teacher and student demands

Teachers submit availability and preferences, and each batch submits its demands through a short survey. Both can be written in plain language and converted by AI. The fitness score combines teacher and student satisfaction and penalizes routines where any single teacher or batch is treated badly.

4. AI assistant with guardrails

Admins, teachers and students can ask about rules, routines, free rooms and makeup slots. Guardrails keep the AI safe:

- Answers come only from department documents and cite their source
- Off-topic questions are refused
- AI output must match a fixed format before it is used
- The AI never edits the routine directly; every suggestion passes the conflict checker and needs human approval
- Role-based access: students ask, teachers request changes, admins approve

5. Maximum classroom utilization

Room use is part of the optimization score. The system matches room capacity to batch size, so a 40-student batch doesn't occupy a 120-seat room, fills idle slots before opening new rooms, and spreads classes evenly across the week. A utilization dashboard shows how busy each room is by day and time, flags underused rooms, and suggests free rooms for makeup or extra classes.

4 Tasks / Work

#	Task
1	Build modules to manage teachers, courses, rooms (type and capacity), batches and time slots
2	Develop the genetic-algorithm scheduler with hard and soft constraints

#	Task
3	Build a RAG pipeline to upload, index and retrieve department rule documents
4	Use AI to extract rules and plain-language preferences into structured constraints, with admin approval
5	Collect teacher preferences and batch-wise student demands, and score satisfaction and fairness
6	Add room utilization to the optimization: capacity matching, idle-slot filling and a room usage dashboard
7	Create the AI assistant for rule questions, routine queries, makeup slots and suggestions
8	Implement guardrails: grounded answers, citations, scope limits, output validation and role-based access
9	Generate teacher-wise, batch-wise and room-wise routines, with manual editing, live conflict checking and PDF export
10	Test with real department data and collect feedback from teachers and students

Planned tools (free)

Python | Google OR-Tools / DEAP | ChromaDB / FAISS | sentence-transformers | LangChain | Pydantic | Free-tier or local LLM (Ollama) | React | Python backend

5 Expected Outcome

A working web system that generates a complete department routine in minutes instead of days. The routine will:

- Have no teacher, room or batch clashes
- Follow department rules automatically, with rules taken from the department's own documents
- Balance teacher and student demands fairly, with satisfaction scores shown for every teacher and batch
- Use classrooms efficiently, with rooms matched to batch size and fewer empty rooms or wasted slots
- Be easy to update when a class is missed or a teacher is absent

Users will get reliable, source-backed answers and suggestions from the AI assistant, while every change stays under human control.

6 Future Scope

Save each semester's routines, admin edits and user feedback. Once enough data builds up, train a machine learning model that learns preferred patterns and guides the scheduler, so each semester's routine is better than the last.